SETU-X

Hybrid-cloud microservices · experimental platform

How network delay, failures and resilience policies shape latency and availability across (simulated) clouds

Parv Agarwal · Junior Research Associate (Technical), AI NLP ML Research Laboratory, IIT Patna · October 2026

63%
of requests are slow with 100-way fan-out when each server is slow 1% of the time [1]

10.4×
rise in tail latency from mismanaging one dependency in a microservice benchmark [2]

10 runs
detected a 10% regression reliably; single runs did not [3]

Research question

With services placed in different (simulated) cloud environments, how do inter-environment delay, dependency failures and client-side resilience policies (timeouts, retries, circuit breakers, cross-environment fallback) change end-to-end latency and success rate? Which of these effects can a local lab establish?

Hypotheses H1 delay propagation · H2 timeouts and breakers · H3 cross-environment fallback · H4 placement · H5 bounded retries

Independent mini-project, not an officially endorsed IIT Patna project.

Known pieces, joined in one runnable lab

Research gap: what the papers leave open

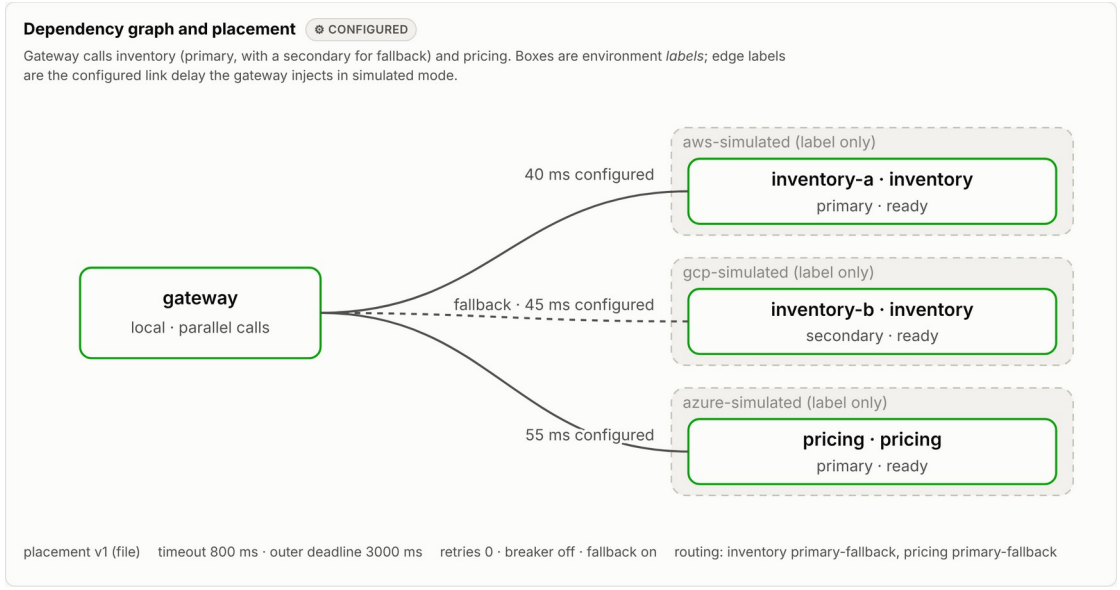
G1 **Placement is studied mostly with models and simulators [4, 5]**
→ SETU-X measures it on running containers with documented link delays

G2 **Fault-injection tools find bugs but prescribe no measurement method [6, 7]**
→ SETU-X couples injection with run-level replication

G3 **Cloud networks vary; even emulators drift from their settings [8–10]**
→ Every simulated result carries explicit validity limits

G4 **Methodology principles often go unmet in cloud studies [3, 11, 12]**
→ SETU-X builds them in by default

Scope: a gap in accessible, disciplined experiment tooling for teaching and small labs, not a claim of novel systems research.



Genuine capture: SETU-X dashboard, simulated mode (Linux build workspace, 26 Oct 2026); controller and runner not drawn.

local	**simulated**	cloud
default; no injected delay	environment labels + injected link delay	off; adapters only validate configuration

- Safety: faults allow-listed, time-bounded, self-expiring; non-root containers on 127.0.0.1
- Cost: no cloud resources, no spend. Ladder simulated → configured → connected → deployed → verified; all providers: not configured

Measured per run, checked by automated gates

How the campaign was run

- Open-loop Poisson load from inside the Compose network; latency from the scheduled send time
- Every run reset and reseeded; run order randomized
- The run is the unit of replication: 3 runs per condition (5 for the baseline), 95% bootstrap CI over runs
- Percentiles per run only (p95 needs ≥ 200 samples, p99 ≥ 1,000), never averaged
- Raw rows and manifest (scenario hash, git commit) re-validated before any analysis

Campaign: 8/8 experiments validated · 64.8 min · one Windows PC (i5-13500, Docker Desktop 29.4.2) · commit bfe122c

Testing gates

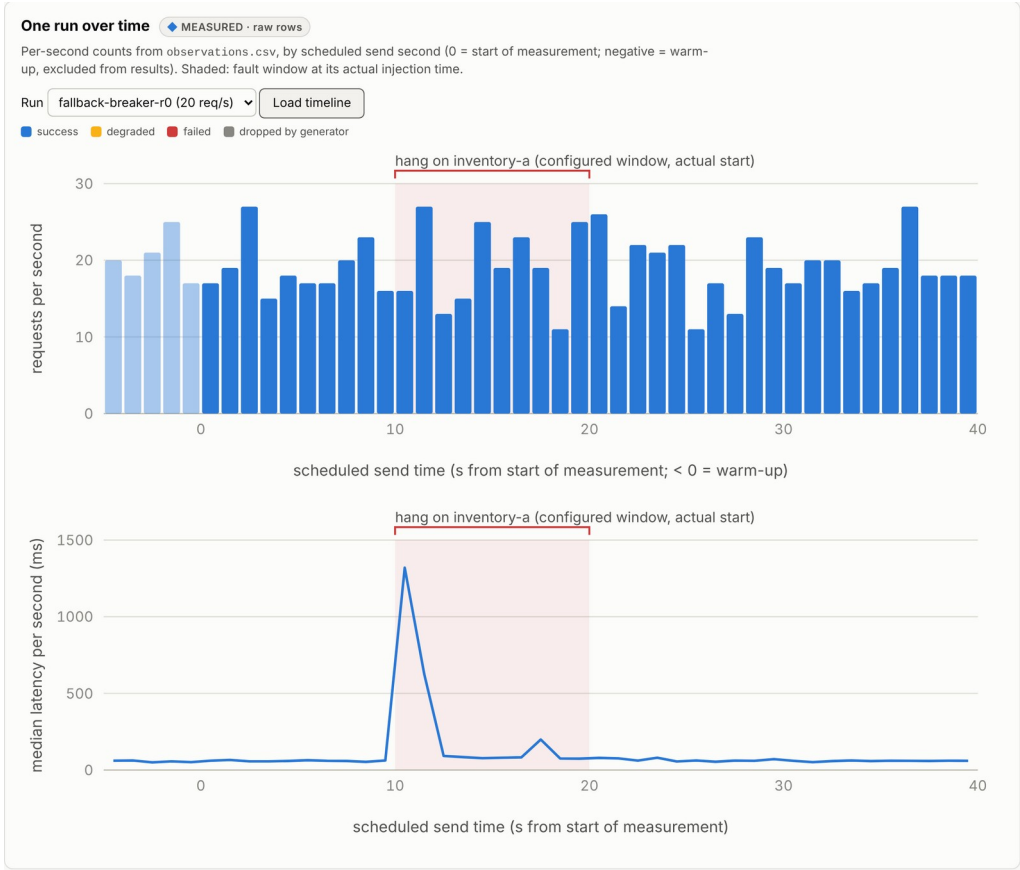
337
unit, contract, integration

10
end-to-end

13
browser (Playwright)

18/18
clean-clone checks, Linux + Windows

GitHub Actions runs the same gates on every push. The pilot caught a 5 s client timeout overriding the scenario's 15 s; fixed, with a regression test, before the campaign.

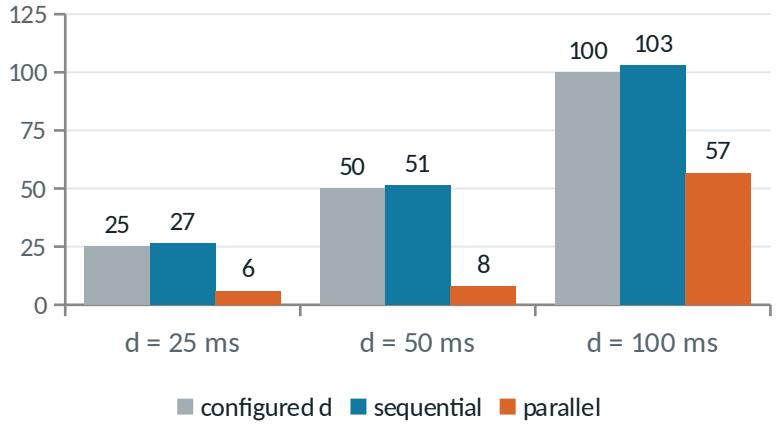


Genuine capture of the Windows records in the dashboard: one Experiment G run (fallback + breaker). The primary hangs for 10 s; every request still succeeds, and latency spikes only until the breaker opens.

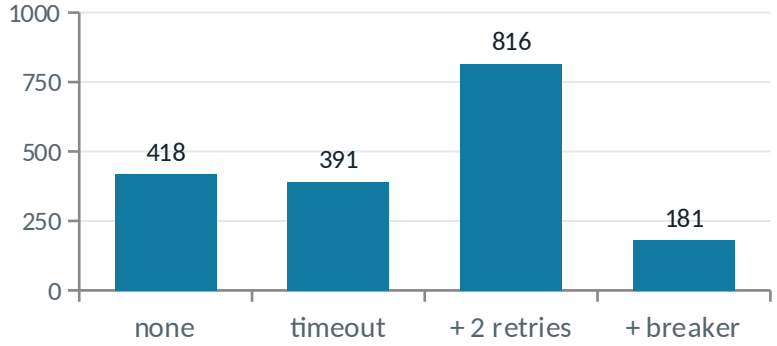
All five hypotheses held here, each with a cost

	Hypothesis	Measured on this machine (medians over runs)	Verdict
H1	Delay adds ≈ d on a sequential path; parallel calls hide it [1]	Sequential +26.6 / 51.3 / 102.9 ms for d = 25 / 50 / 100 ms; parallel +6.0 / 8.0 / 56.7 ms	**Supported**
H2	Timeouts bound latency; breakers cut load and fail fast [13]	Failed requests 10.0 s → 0.81 s with a timeout. Breaker: 78% fewer calls, fails in 17 ms, but recovery 1.0 s → 5.2 s	**Supported, slower recovery**
H3	Fallback to another environment raises success, at a latency cost	Success 0.74 → 1.00 (+0.257 [0.251, 0.266]); p95 +749 ms; with a breaker p95 239–524 ms	**Supported**
H4	Placement helps only when it moves the critical path	Moving pricing saves 31.3 ms [29.0, 34.3]; moving inventory 0.4 ms [−3.7, 2.6]	**Supported**
H5	Retries reach 1 − p^(1+r) and multiply calls [1]	Success 0.50 / 0.77 / 0.88 vs predicted 0.5 / 0.75 / 0.875; 1.00 / 1.48 / 1.75 calls per request	**Supported***
E	Where this host saturates (exploratory)	Gateway at its 0.5-CPU limit by 40 req/s (46.4–47.1% CPU); at 80 req/s goodput collapses to 0.5–9.2 req/s	**Observed**

H1 · median latency shift vs d = 0 (ms)



H2 · calls to the hung service (median per run)



* The injected errors are independent by construction, so H5 checks the retry logic, not real-world failure correlation. [a, b] = 95% bootstrap CI over runs; 3 runs per condition; local and simulated modes on one Windows PC.

What transfers, what does not, what is next

What the lab shows

- The critical path decides: delay and placement matter only where they sit on it
- Each resilience policy trades something: errors for bounded latency, recovery time for less load, latency for success
- Without admission control, overload collapses goodput instead of levelling it off

What it cannot show

- Provider numbers: one host, constant injected delay, application-level faults
- Independent failure domains: both replicas share one kernel
- Fine tails: p99 is reportable only for the baseline; 3 runs per condition

Next

- Same scenarios on real providers (adapters target ECS Fargate, Azure Container Apps, Cloud Run)
- Jitter and loss models, admission control, more runs
- Code, data and CI (Apache-2.0; docs and results CC BY 4.0):

github.com/Parv-01/setu-x

References (full bibliography with DOIs: docs/references.bib)

[1] J. Dean, L. A. Barroso. The tail at scale. CACM, 2013.
[2] Y. Gan et al. An open-source benchmark suite for microservices… ASPLOS 2019.
[3] S. Eismann et al. Microservices: a performance tester's dream or nightmare? ICPE 2020.
[4] A. Brogi, S. Forti. QoS-aware deployment of IoT applications through the fog. IEEE IoT J., 2017.
[5] M. Mota-Cruz et al. Optimizing microservices placement in the cloud-to-edge continuum. arXiv:2412.01412, 2024.
[6] V. Heorhiadi et al. Gremlin: systematic resilience testing of microservices. ICDCS 2016.
[7] C. S. Meiklejohn et al. Service-level fault injection testing. SoCC 2021.
[8] P. Leitner, J. Cito. Patterns in the chaos: performance variation in public IaaS clouds. ACM TOIT, 2016.
[9] A. Uta et al. Is big data performance reproducible in modern cloud networks? NSDI 2020.
[10] A. Jurgelionis et al. An empirical study of NetEm network emulation functionalities. ICCCN 2011.
[11] A. V. Papadopoulos et al. Methodological principles for reproducible performance evaluation in cloud computing. IEEE TSE, 2021.
[12] T. Hoefler, R. Belli. Scientific benchmarking of parallel computing systems. SC 2015.
[13] F. Montesi, J. Weber. Circuit breakers, discovery, and API gateways in microservices. arXiv:1609.05830, 2016.